

```

import random

cities = ['A', 'B', 'C', 'D', 'E']

dist = {
    ('A','B'): 2, ('A','C'): 5, ('A','D'): 7, ('A','E'): 6,
    ('B','C'): 3, ('B','D'): 6, ('B','E'): 4,
    ('C','D'): 4, ('C','E'): 5, ('D','E'): 3
}

def distance(route):
    d = 0
    for i in range(len(route)):
        a = route[i]
        b = route[(i + 1) % len(route)]
        d += dist.get((a,b), dist.get((b,a)))
    return d

# Initial population
pop = []
for _ in range(20):
    r = cities.copy()
    random.shuffle(r)
    pop.append(r)

# GEA iterations
for _ in range(100):

    # Selection
    pop.sort(key=distance)
    parents = pop[:10]

    # Crossover + Mutation
    new_pop = parents.copy()

    while len(new_pop) < 20:
        p = random.choice(parents)
        child = p.copy()

        # Mutation
        i, j = random.sample(range(5), 2)
        child[i], child[j] = child[j], child[i]

        new_pop.append(child)

    pop = new_pop

# Best solution

```

```
best = min(pop, key=distance)

print("Best Route:", best + [best[0]])
print("Minimum Distance:", distance(best))
```

```
# GEA iterations
for _ in range(100):

    # Selection
    pop.sort(key=distance)
    parents = pop[:10]

    # Crossover + Mutation
    new_pop = parents.copy()

    while len(new_pop) < 20:
        p = random.choice(parents)
        child = p.copy()

        # Mutation
        i, j = random.sample(range(5), 2)
        child[i], child[j] = child[j], child[i]

        new_pop.append(child)

    pop = new_pop

# Best solution
best = min(pop, key=distance)

print("Best Route:", best + [best[0]])
print("Minimum Distance:", distance(best))
```

```
... Best Route: ['A', 'B', 'C', 'D', 'E', 'A']
Minimum Distance: 18
```